Interfaces

Orientação a Objetos – DCC025

2012.3

Interface – Declaração

- Declaração de Interfaces
  - Inicia-se com a palavra-chave “interface” e contém somente constantes e métodos do tipo “abstract”.
  - Diferentemente das classes, TODOS os membros de interface devem ser public e as interfaces não podem especificar nenhum detalhe de implementação como declarações de métodos concretos

Interface – Declaração

- Declaração de Interfaces
  - TODOS os métodos declarados em uma interface são implicitamente métodos public abstract e todos os campos são public, static e final.

- OBS
  - Interfaces não contém atributos, apenas assinaturas de métodos
  - Contudo, constantes podem ser declaradas na interface

```
1 public interface IUsaCombustivel {
2
3     public int valor;
4
5 }
6
```

The blank final field valor may not have been initialized

```
1 public interface IUsaCombustivel {
2
3     int valor = 0;
4
5 }
```

Interfaces

```
interface Conta {
    void deposita(double valor);
    void saca(double valor);
}
```

```
class ContaPoupanca implements Conta {
    public void deposita(double valor) {
        // implementacao
    }
    public void saca(double valor) {
        // implementacao
    }
}
```

```
class ContaCorrente implements Conta {
    public void deposita(double valor) {
        // implementacao
    }
    public void saca(double valor) {
        // implementacao
    }
}
```

```java
interface Conta {
    void deposita(double valor);
    void saca(double valor);
}
```

Todos os métodos de uma interface são “Abstract” e “Public”

```java
// Esta classe não compila porque ela não implementou o método saca()
class ContaCorrente implements Conta {
    public void deposita(double valor) {
        // implementacao
    }
}
```

As classes concretas que implementam a interface são obrigadas a implementar todos os métodos declarados na interface.

Erro de compilação caso isso não seja feito

Interfaces

```
interface Coins {
  int
    PENNY = 1,
    NICKEL = 5,
    DIME = 10,
    QUARTER = 25,
    DOLAR = 100;
}

class SodaMachine implements Coins {
  int price = 3*QUARTER;
  // ...
}
```

A interface Coins define um conjunto de constantes. As classes que implementam tal interface irão, obrigatoriamente, possuir as mesmas constantes.

Neste caso, não é preciso declarar novamente tais constantes na classe SodaMachine

Diferenças entre Classes Abstratas e Interfaces

Diferença Classes Abstratas e Interfaces

- Algumas Diferenças
  - A interface obrigatoriamente não tem um “corpo” associado. Para que uma classe seja abstrata basta que ela seja assim declarada, mas a classe pode incluir atributos de objetos e definição de métodos, públicos ou não. Na interface, apenas métodos públicos podem ser declarados — mas não definidos. Da mesma forma, não é possível definir atributos — apenas constantes públicas.

Diferença Classes Abstratas e Interfaces

- Algumas Diferenças
  - Enquanto uma classe abstrata é “estendida” (palavra chave extends) por classes derivadas, uma interface Java é “implementada” (palavra chave implements) por outras classes.

Diferença Classes Abstratas e Interfaces

- Diferença
  - A diferença essencial entre classes abstratas e interfaces é que uma classe herdeira somente pode herdar de uma única classe (abstrata ou não), enquanto que qualquer classe pode implementar várias interfaces simultaneamente.

Diferença Classes Abstratas e Interfaces

- Diferença
  - Interfaces são um mecanismo simplificado de implementação de herança múltipla em Java, permitindo que mais de uma interface determine os métodos que as que uma classe herdeira deve implementar.
    - OBS: lembre-se que Java não suporta herança múltipla. Logo, uma classe herdeira (uma subclasse) só pode herdar de uma única superclasse.

Diferença Classes Abstratas e Interfaces

- Diferença
  - Interface é dita ser uma classe abstrata "pura"
  - **Toda interface deve ser vista como uma classe abstrata (o contrário é falso)**

Quando usar Interfaces?

- Quando projetar com interfaces
  - Quando se deseja apoiar polimorfismo sem se comprometer com uma particular hierarquia de classes. Se uma superclasse AC é usada sem uma interface, qualquer nova solução polimórfica precisa ser uma subclasse de AC (se não for, será cópia de código)

Implementação de herança

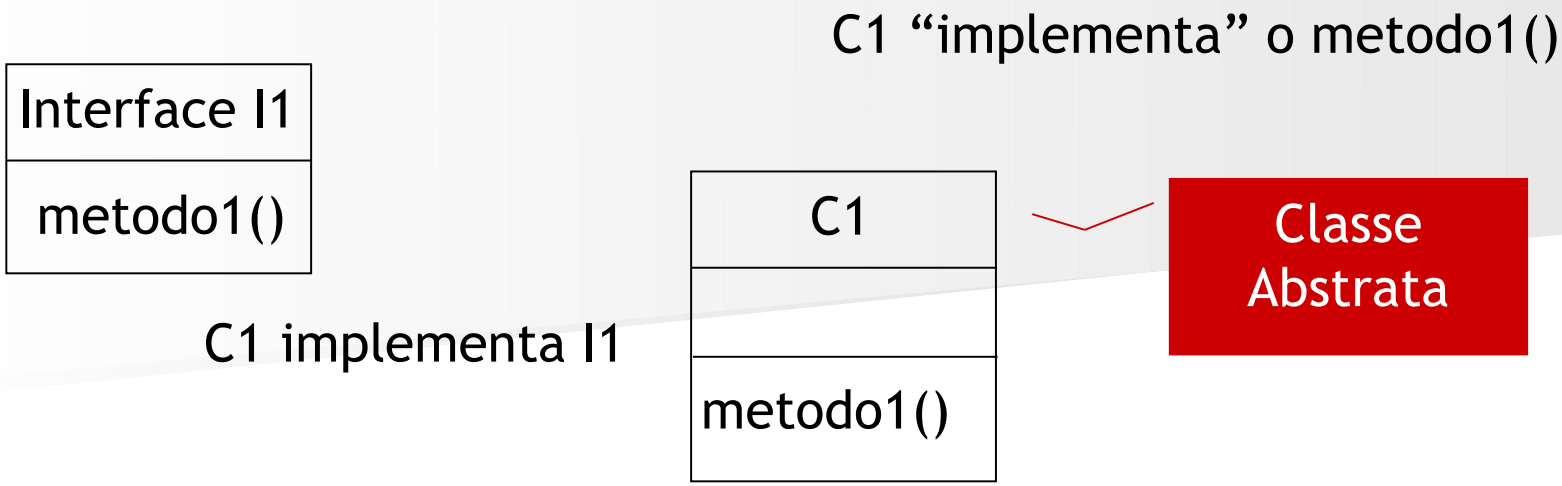
Implementação de outra classe com o mesmo método - cópia de código (erro)

Quando Usar Interfaces

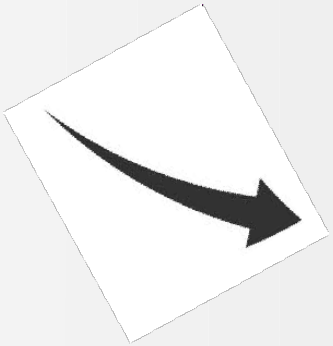
- Regra Geral
  - Se existe uma hierarquia de classes com uma superclasse abstrata C1, considere fazer uma interface I1 que corresponde à assinatura do método público de C1, e depois declare C1 para implementar a interface I1.

Quando Usar Interfaces

- Ponto de flexibilização para casos futuros
  - Outras classes podem implementar a interface I1
  - Se necessário, classes podem herdar de C1



Quando Usar Interfaces



Interface I1

C1
metodo1()

Posso ter uma classe que não nada a ver com C1 (em termos de herança), mas que precisa implementar os mesmos métodos

Posso ter uma classe que precise herdar de C1

OBS: uma classe que herde de CI não precisa, obrigatoriamente, implementar o método1()
Isto porque essa suposta classe herdeira de C1
já está herdando o metodo1()

```java
public interface IFuncionario {

    int calcularSalario();

}
```

```java
public class Funcionario implements IFuncionario{

    public int calcularSalario() {
        // TODO Auto-generated method stub
        return 0;
    }

}
```

```java
public class Gerente extends Funcionario {

}
```

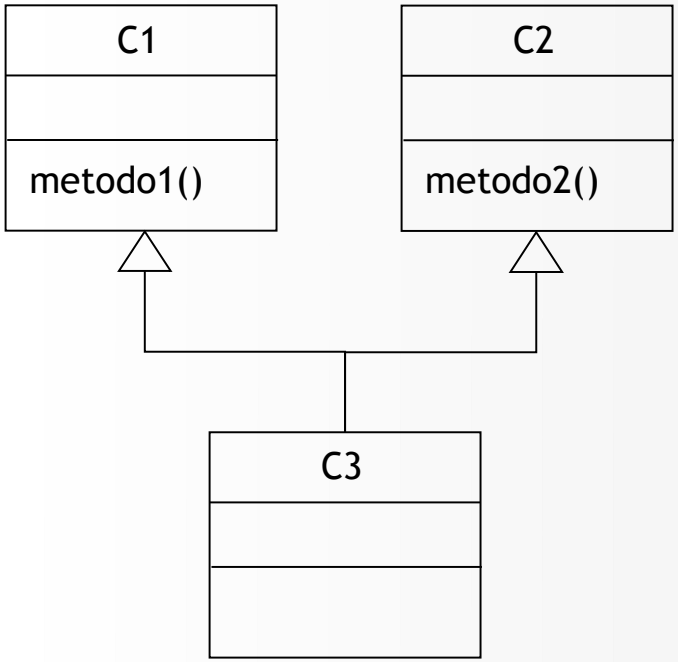
Interfaces - Vantagens

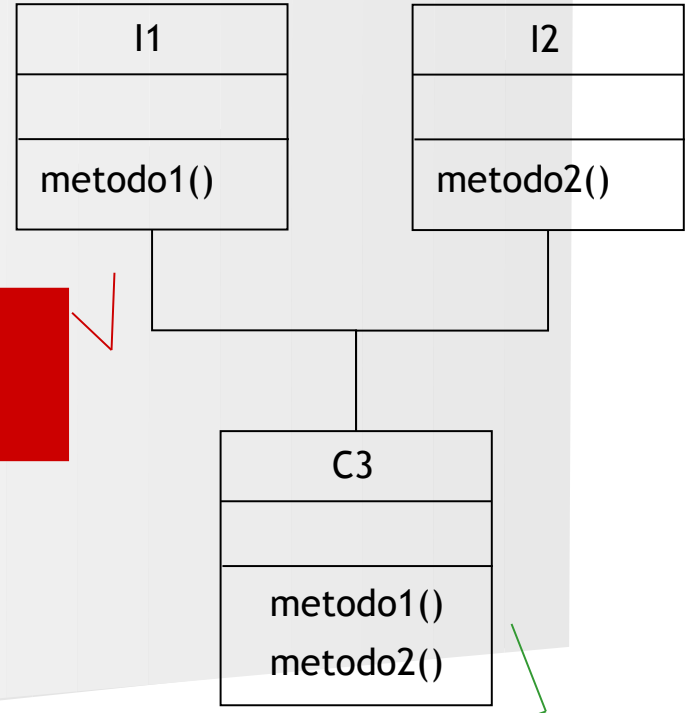
- Algumas vantagens
  - Padronização de assinatura de métodos
  - Garantir que certas classes implementem certos métodos
  - Possibilidade de implementar diversas interfaces

Interfaces - Vantagens

- Algumas vantagens
  - OBS: Embora OO apresente o conceito de herança múltipla (possibilidade de uma classe herdar de duas ou mais classes), Java não oferece suporte ao mesmo. Contudo, uma forma de fazer uma classe “herdar” coisas de mais de uma classe é através do mecanismo de interfaces.

Interfaces





Métodos abstratos

Herança múltipla: a classe C3 herda das classes C1 e C2.

Por ser uma relação de interface, obrigatoriamente, C3 deve implementar os métodos 1 e 2

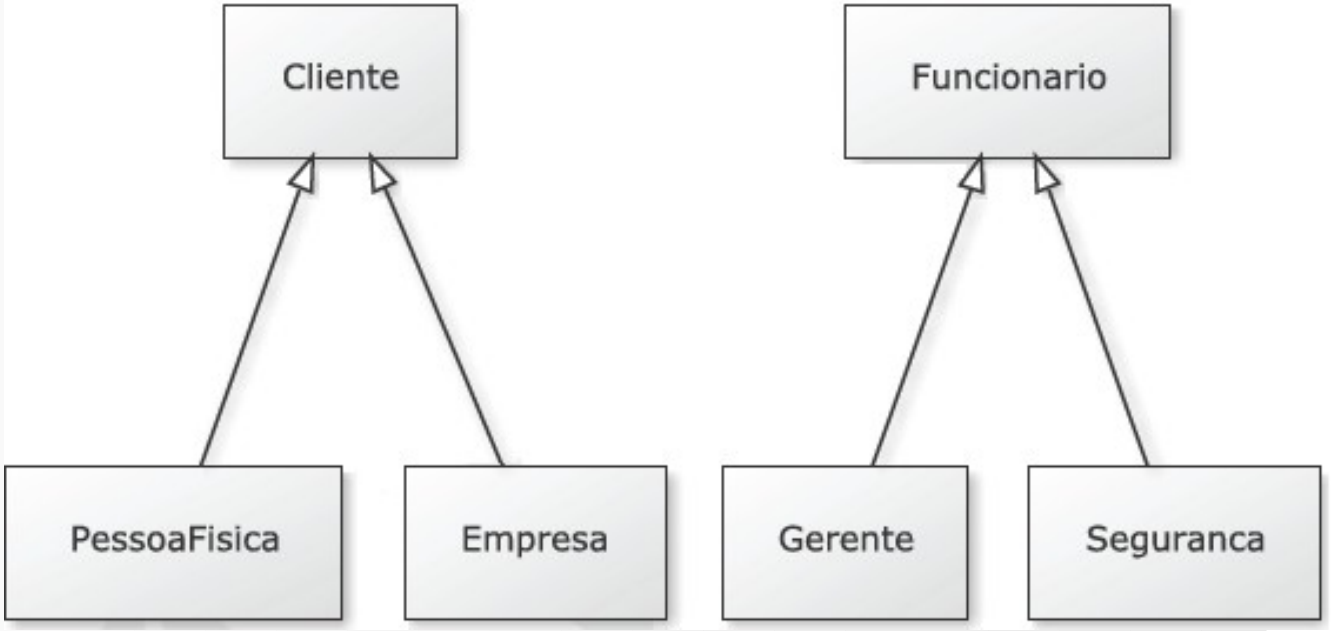
Interfaces e Herança

- Tema de discussão
  - Uso de interfaces ou uso de herança
  - A grosso modo, priorizar a utilização de interfaces permite que alterações pontuais em determinados trechos do código fonte sejam feitas mais facilmente pois diminui as ocorrências de efeitos colaterais indesejados no resto da aplicação.

Interfaces e Herança

- Tema de discussão
  - Por outro lado, priorizar a utilização de herança pode diminuir a quantidade de código escrito no início do desenvolvimento de um projeto.

Interfaces e Herança



Vamos supor que gerentes e empresas podem acessar o sistema de um banco através de um usuário e de uma senha.

Proposta inicial: implementar um único método para fazer a autenticação (validação o usuário de senha)

Interfaces e Herança

```
class AutenticadorDeUsuario {
    public boolean autentica(??? u) {
        // implementação
    }
}
```

Pergunta: qual seria o tipo de parâmetro de método?

Um objeto do tipo Empresa?
Um objeto do tipo Gerente?

OBS: Não há como implementar polimorfismo utilizando um método geral.

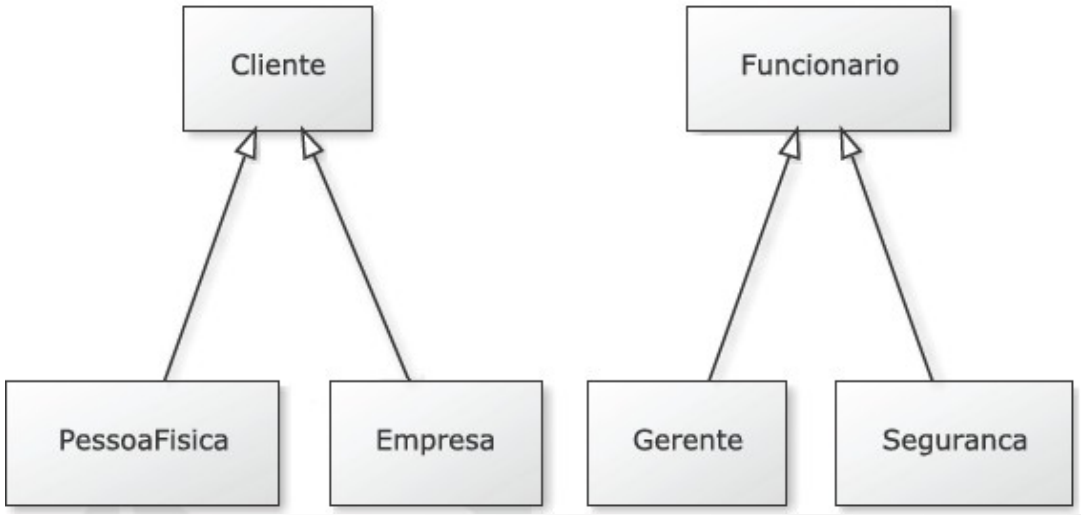
Quais as consequências disso?

Interfaces e Herança

```
class AutenticadorDeUsuario {
    public boolean autentica(??? u) {
        // implementação
    }
}
```

```
class AutenticadorDeUsuario {
    public boolean autentica(Empresa u)
}
```

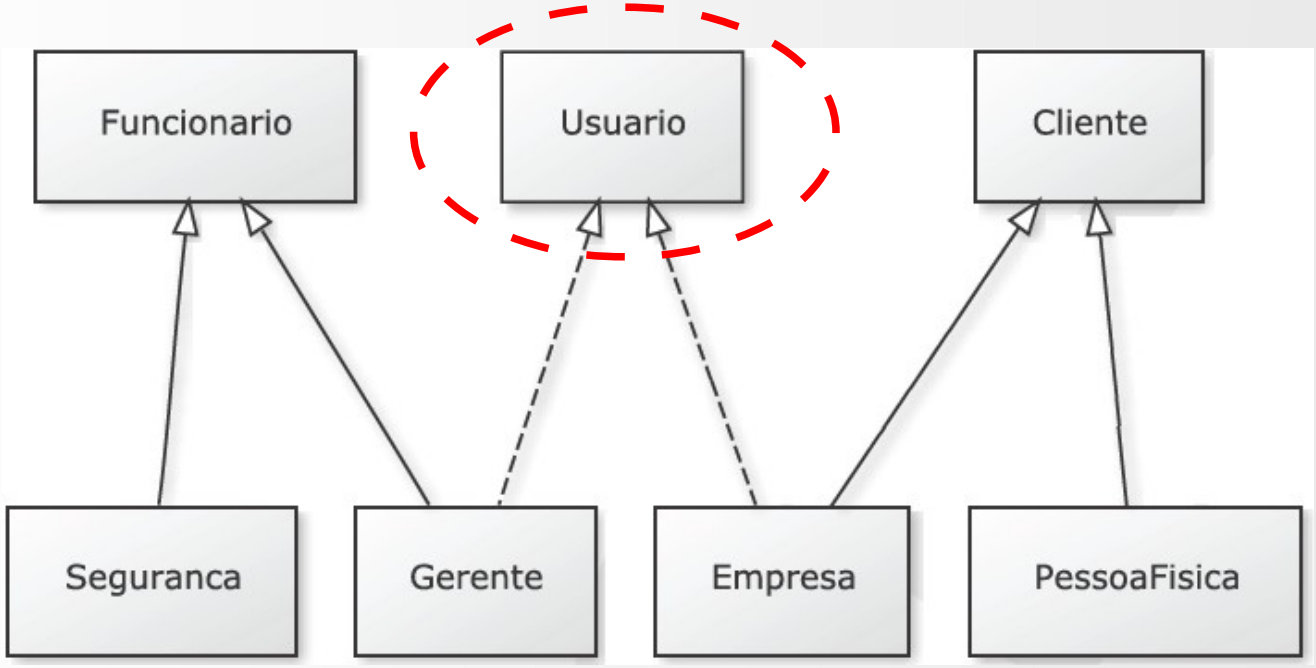
```
class AutenticadorDeUsuario {
    public boolean autentica(Gerente u)
}
```



OBS: Não há polimorfismo entre objetos da classe Empresa e objetos da classe Gerente

Para obter polimorfismo entre os objetos dessas duas classes somente com herança, deveríamos colocá-las na mesma árvore de herança. Mas, isso não faz sentido pois uma empresa não é um funcionário e o gerente não é cliente.

Interfaces e Herança

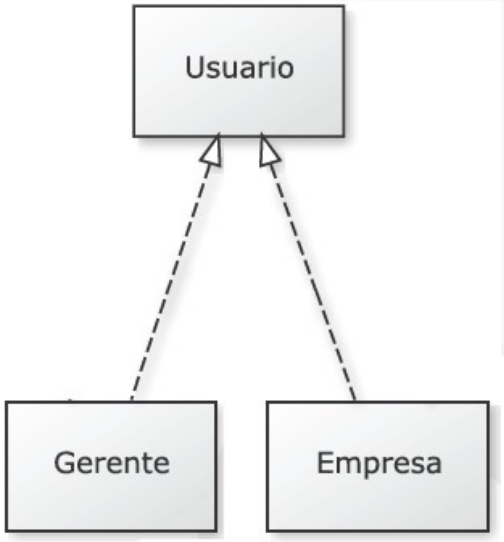


A solução é usar interfaces para obter polimorfismo entre objetos da classe Gerente e da classe Empresa

Interfaces e Herança

```
class AutenticadorDeUsuario {
    public boolean autentica(Usuario u) {
        // implementação
    }
}
```

conseguimos definir o que o método AUTENTICA() deve receber um parâmetro para trabalhar tanto com gerentes quanto com empresas. Ele deve receber um parâmetro do tipo USUARIO.



Pergunta: a Interface Usuário poderia ser uma classe?

Interfaces

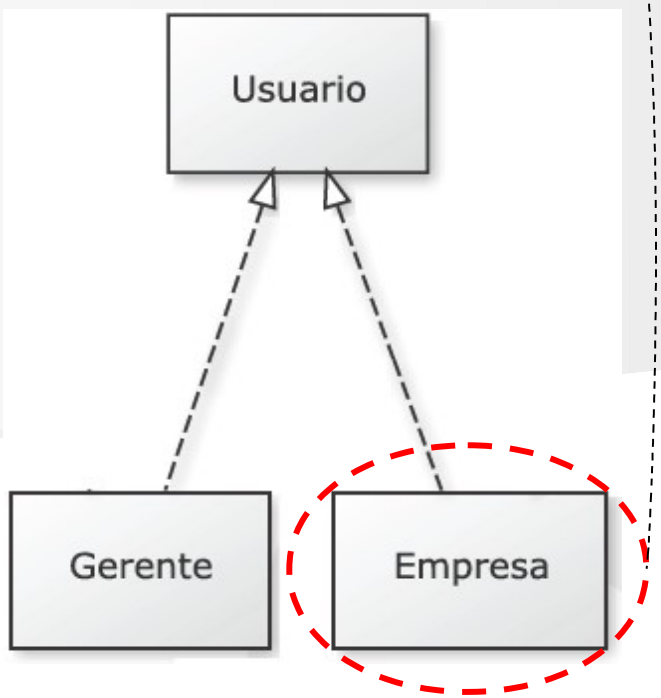
- Suponhamos
  - Classe A implementa interface B
  - Neste caso, posso tratar objetos do tipo A como se fossem do tipo B

Interfaces

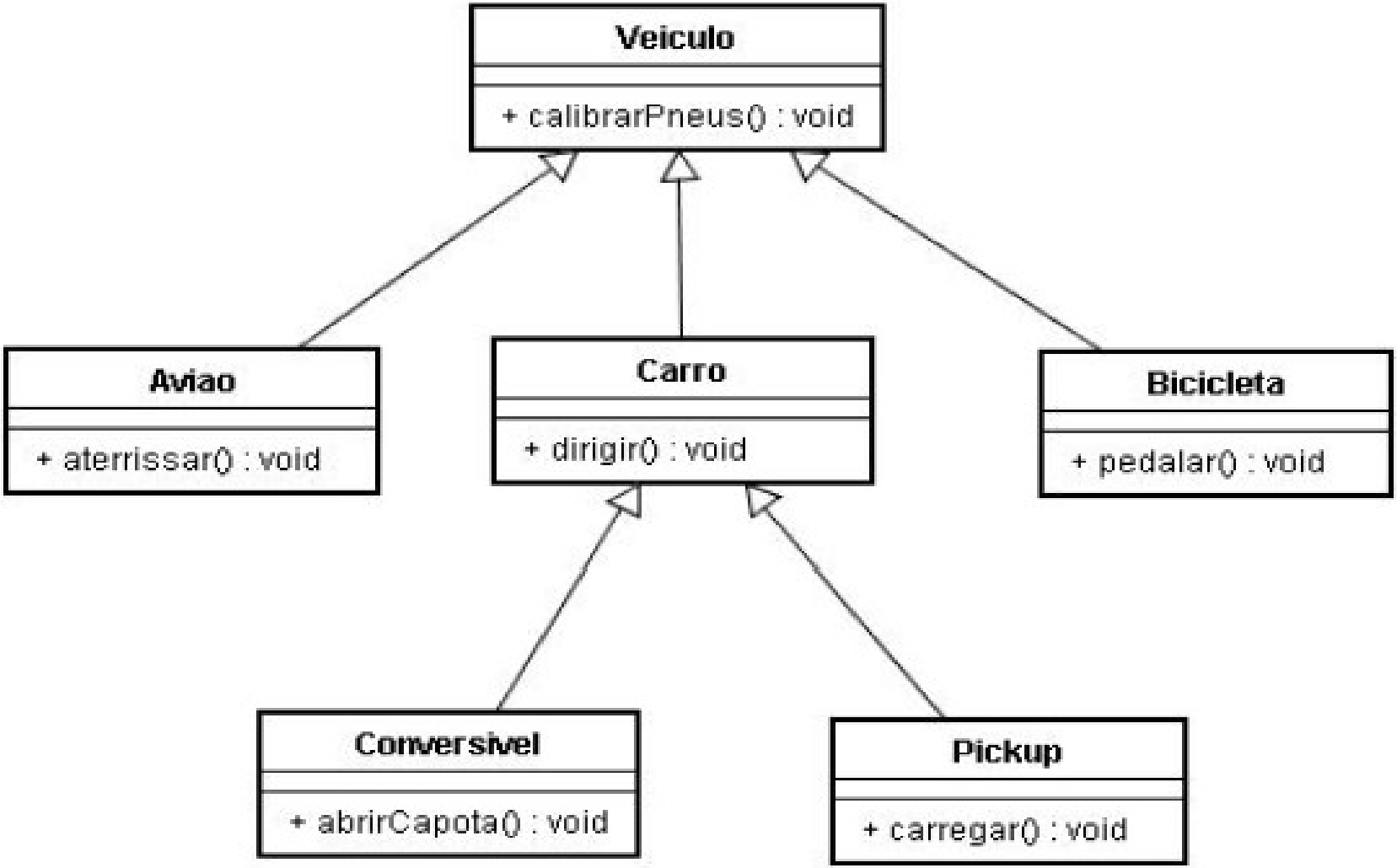
- Suponhamos
  - No exemplo anterior
    - Posso tratar objetos do tipo Gerente (ou Empresa) como se fosse Usuário
    - Posso dizer que “um Gerente é um Usuário”
    - Isso permite polimorfismo
    - Podemos fazer Usuario u = new Gerente()

Interfaces e Herança

```
class AutenticadorDeUsuario {
    public boolean autentica(Usuario u) {
        // implementação
    }
}
```



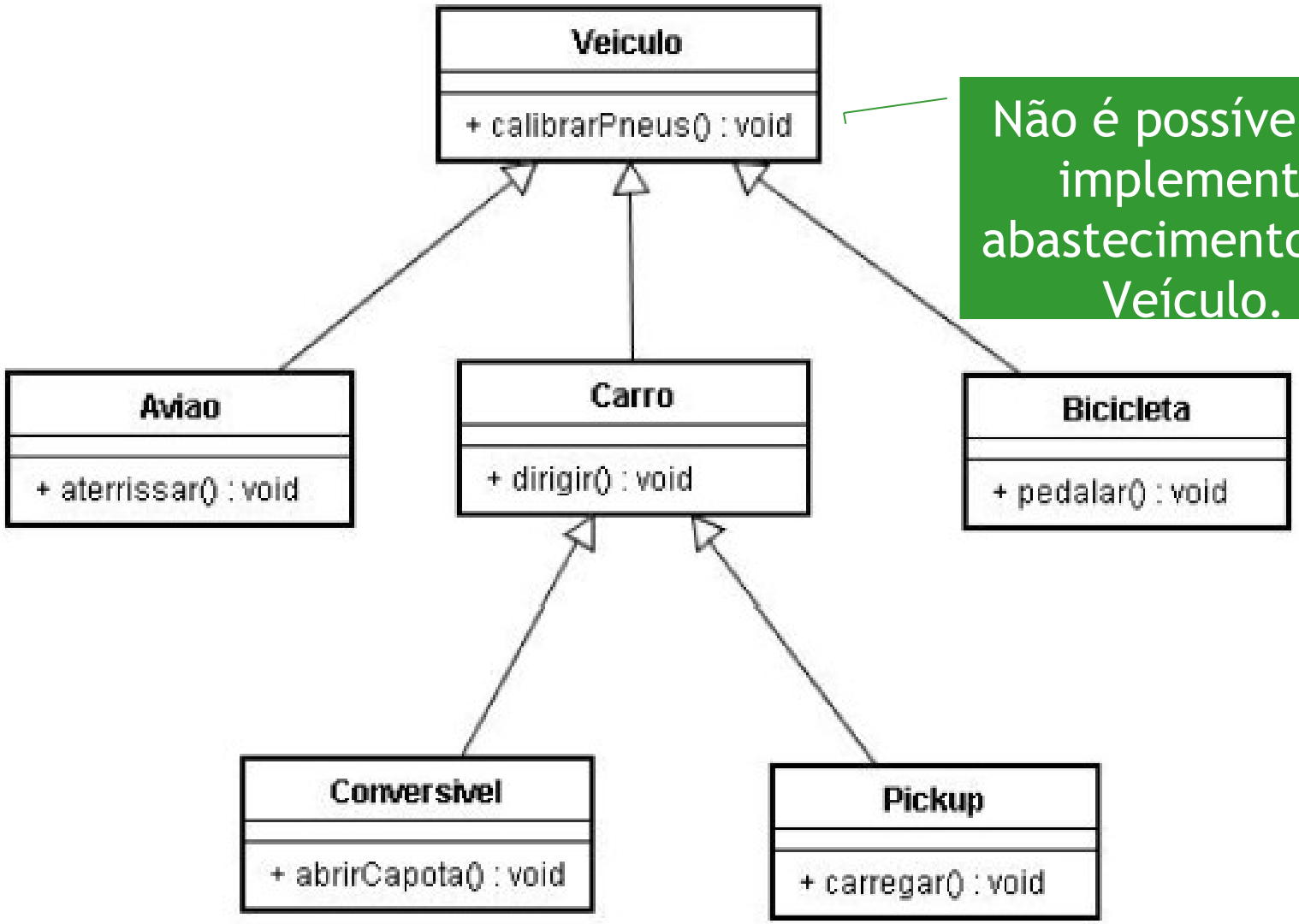
Aplicação de Interfaces



Aplicação de Interfaces

- Considerações (diagrama anterior)
  - Todos os meios de transporte listados na hierarquia possuem pneus. Por isso, o método calibrarPneus() foi declarado no topo da hierarquia, sendo, dessa forma, herdado pelas demais subclasses;
  - Apenas os aviões aterrissam. Logo, o método aterrissar() foi declarado na subclasse Aviao.

Aplicação de Interfaces



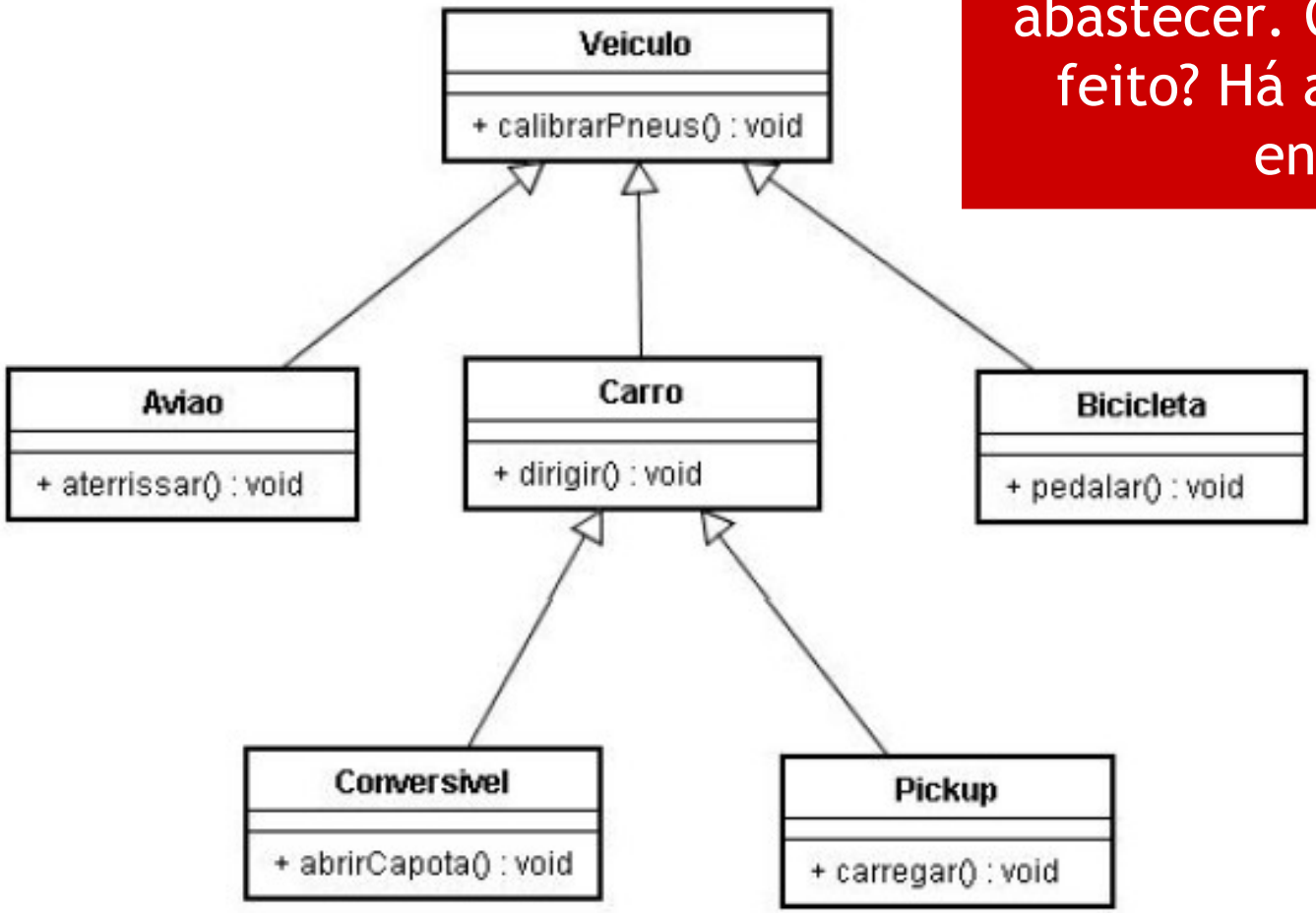
Não é possível definir uma implementação para abastecimento nesta classe Veículo. Porque?

Aplicação de Interfaces

- Mais considerações
  - Quase todos os veículos da hierarquia anterior podem ser abastecidos com combustível. Entretanto, se fosse definida uma implementação para tal na classe Veiculo, ela seria herdada por Bicicleta, que não pode ser abastecida;

Aplicação de Interfaces

Suponha que seja necessário implementar um método para abastecer. O que poderia ser feito? Há algum problema envolvido?



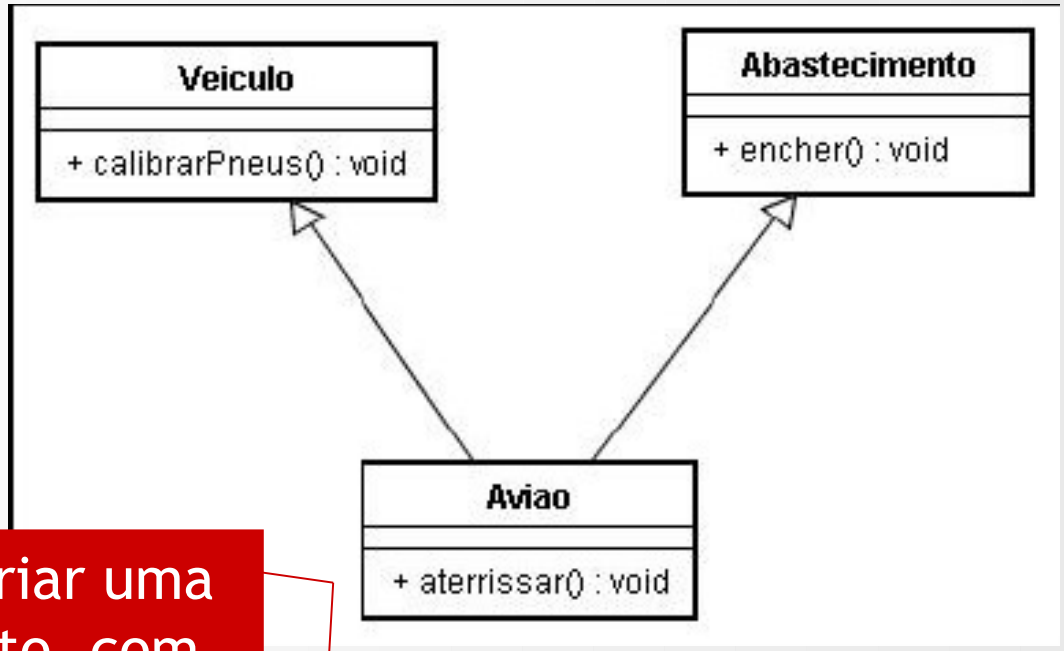
Aplicação de Interfaces

- Considerações
  - Definir métodos distintos para abastecimento nas classes Aviao e Carro introduziria uma “redundância” indesejável.
  - Logo, como resolver este dilema?

Aplicação de Interfaces

- Considerações
  - Não podemos declarar um método para abastecimento em Veículo, pois nem todas as subclasses podem ser abastecidas. Também não podemos declarar um método para cada classe que necessite de abastecimento, pois isso gera redundância de código.

Aplicação de Interfaces



Uma solução seria criar uma classe Abastecimento, com um método para abastecer. Assim, toda classe que necessite de abastecimento, iria herdar da classe Abastecimento

Porque isso é problemático em Java?

```
public class Aviao extends Veiculo implements IUsaCombustivel
{
  private double capcTanque=5000.00;
  private double totComb=2000.00;
  public void aterrisar(){};
  public double encher(TipoComb tipo,double qtd)
  {
    if(tipo!=TipoComb.Querosene)
      return qtd;
    double falta=capcTanque-totComb;
    if(qtd>falta)
    {
      totComb=capcTanque;
      return qtd-falta;
    }
    else
    {
      totComb+=qtd;
      return 0.0;
    }
  }
}
```

```
public class Carro extends Veiculo implements IUsaCombustivel
{
  private double capcTanque=50.00;
  private double totComb=20.00;
  public void dirigir(){};
  public double encher(TipoComb tipo,double qtd)
  {
    if(tipo!=TipoComb.Alcool &&
      tipo!=TipoComb.Gasolina)
      return qtd;
    double falta=capcTanque-totComb;
    if(qtd>falta)
    {
      totComb=capcTanque;
      return qtd-falta;
    }
    else
    {
      totComb+=qtd;
      return 0.0;
    }
  }
}
```

Estudo de Caso: criando e utilizando interfaces

- Suponha que a empresa do exemplo de folha de pagamento deseja realizar operações de contabilidade em um único aplicativo de contas a pagar (calcular vencimentos de funcionarios também calcula pagamento devido a cada fatura – contas de mercadorias adquiridas).
- Poderíamos calcular coisas tão diferentes polimorficamente?

Estudo de Caso: criando e utilizando interfaces

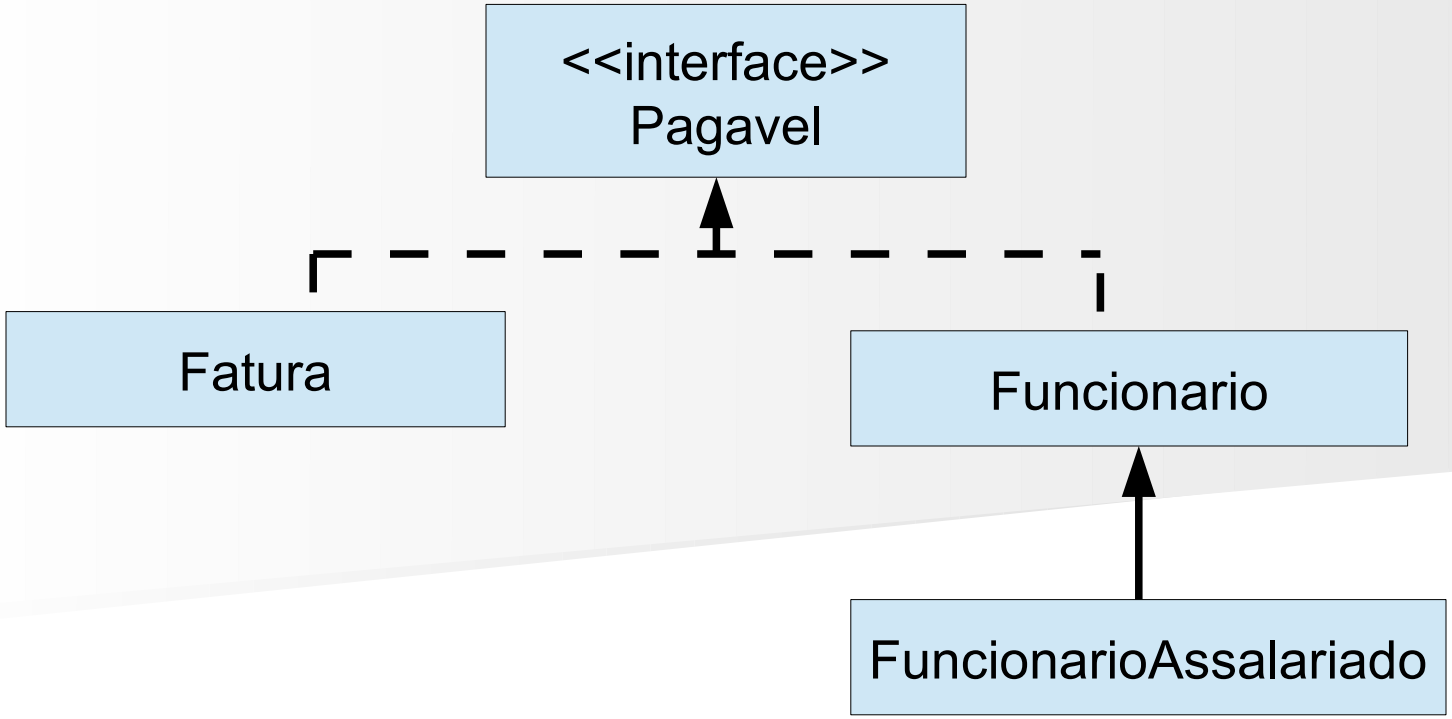
- Suponha que a empresa do exemplo de folha de pagamento deseja realizar operações de contabilidade em um único aplicativo de contas a pagar (calcular vencimentos de funcionarios também calcula pagamento devido a cada fatura – contas de mercadorias adquiridas).
- Poderíamos calcular coisas tão diferentes polimorficamente?
- As interfaces do Java oferecem essa capacidade de exigir que classes não relacionadas implementem um conjunto de métodos comuns
  - por exemplo o método que calcula a quantia de um pagamento.

Estudo de Caso: criando e utilizando interfaces

- Para utilizar uma interface, a classe concreta deve especificar que implementa (**implements**) a interface e deve declarar todos os métodos da interface
- Vamos criar a interface *Pagavel*
  - Método getValorPagamento() que retorna um valor double a ser pago para um objeto de qualquer classe que implementa a interface
  - Este método é uma versão geral do método ganhos() do exemplo que usamos classe abstrata
  - Obs: ao declarar um método em uma interface escolha um nome de método que descreva o propósito do método de forma geral pois pode ser implementado por muitas classes não relacionadas

Estudo de Caso: criando e utilizando interfaces

- A classe Funcionario é modificada para implementar a interface *Pagavel* e apresentamos uma classe Fatura que também implementa a mesma interface.



Estudo de Caso: criando e utilizando interfaces

- Obs:
  - Métodos de Interface sempre são public e abstract e escrever isso é redundante
  - Campos em uma interface sempre são final e static e também é redundância escrever

```
public interface Pagavel
{
    double getValorPagamento();//calcula pagamento;
    //nenhuma implementacao
}
```

Estudo de Caso: criando e utilizando interfaces

- Modificando a classe Funcionario
  - Inserimos implements Pagavel
  - A classe não implementa o método e deve ser mantida abstrata para evitar erro de compilacao

```
//superclasse abstrata Funcionario
public abstract class Funcionario implements Pagavel
{
    private String nome;
    private String sobrenome;
    private String id;
```

Estudo de Caso: criando e utilizando interfaces

- Modificando a classe FuncionarioAssalariado
  - Apos implementar Pagavel devemos renomear o metodo ganhos para getValorPagamento nas subclasses de Funcionario

```
//sobrescreve o metodo abstrato
public double getValorPagamento(){
    return getSalarioSemanal();
}
```

Estudo de Caso: criando e utilizando interfaces

```
//classe Fatura que implementa interface Pagavel
public class Fatura implements Pagavel
{
    private String numero;
    private String descricao;
    private int quantidade;
    private double precoItem;
    //construtor com quatro argumentos
    public Fatura(String numero, String descricao,
        int quantidade, double precoItem){
        this.numero = numero;
        this.descricao = descricao;
        setQuantidade(quantidade);//valida e armazena
        setPrecoItem(precoItem);
    }
```

Estudo de Caso: criando e utilizando interfaces

```
//configura numero da peca
public void setNumero(String numero){
    this.numero = numero;
}
//retorna numero da peca
public String getNumero(){
    return numero;
}
//configura descricao da peca
public void setDescricao(String descricao){
    this.descricao = descricao;
}
//retorna descricao
public String getDescricao(){
    return descricao;
}
```

Estudo de Caso: criando e utilizando interfaces

```
//valida e configura quantidade
public void setQuantidade(int quantidade){
    this.quantidade = (quantidade < 0) ? 0 : quantidade;
}
//retorna quantidade
public int getQuantidade(){
    return quantidade;
}
//valida e configura preco por item
public void setPrecoItem(double preco){
    this.precoItem = (preco < 0.0) ? 0.0 : preco;
}
//retorna preco por item
public double getPrecoItem(){
    return precoItem;
}
```

Estudo de Caso: criando e utilizando interfaces

```
    //retorna representacao string do objeto
    public String toString(){
        return String.format("%s: \n%s: %s(%s) \n%s: %d \n%s: R$%,.2f",
            "fatura", "numero da peca", getNumero(), getDescricao(),
            "quantidade", getQuantidade(), "preco por item",
            getPrecoItem());
    }
    //metodo requerido pela interface Pagavel
    public double getValorPagamento(){
        return getQuantidade()*getPrecoItem();//calcula custo total
    }
}
```

Estudo de Caso: criando e utilizando interfaces

- Quando uma classe implementa uma interface a mesma relação *é um* da herança se aplica
- Podemos dizer que:
  - Funcionario é um Pagavel
  - FuncionárioAssalariado é um Pagável
  - Fatura é um Pagavel

Estudo de Caso: criando e utilizando interfaces

- Utilizando a interface Pagavel objetos de classes completamente desconectadas (Faturas e Funcionarios) podem ser processados polimorficamente
- Crie uma classe de Teste

Estudo de Caso: criando e utilizando interfaces

```
public class TesteInterfacePagavel
{
    public static void main(String[] args){
        //cria array de quatro elementos Pagavel
        Pagavel[] objetosPagaveis = new Pagavel[4];

        //preenche o array com objetos que implementam
        //a interface pagavel
        objetosPagaveis[0] = new Fatura("01234", "banco", 2, 375.00);
        objetosPagaveis[1] = new Fatura("56789", "pneu", 4, 79.95);
        objetosPagaveis[2] = new FuncionarioAssalariado(
            "Joao", "da Silva", "111-111-111", 800.00);
        objetosPagaveis[3] = new FuncionarioAssalariado(
            "Maria", "Souza", "888-888-888", 1200.00);
```

Estudo de Caso: criando e utilizando interfaces

```
System.out.println("Faturas e funcionarios processados polimorficamente:\n");


//processa genericamente cada elemento no array
for(Pagavel pagAtual : objetosPagaveis){
    //gera saida do objeto atual
    System.out.printf("%s \n%s: R$%,.2f\n\n", pagAtual.toString(),
        "valor pagamento", pagAtual.getValorPagamento());
}
```

```
Faturas e funcionarios processados polimorficamente:

fatura:
numero da peca: 01234(banco)
quantidade: 2
preco por item: R$375,00
valor pagamento: R$750,00

fatura:
numero da peca: 56789(pneu)
quantidade: 4
preco por item: R$79,95
valor pagamento: R$319,80

Funcionario Assalariado: Joao da Silva
Identidade: 111-111-111
salario semanal: : R$800,00
valor pagamento: R$800,00

Funcionario Assalariado: Maria Souza
Identidade: 888-888-888
salario semanal: : R$1.200,00
valor pagamento: R$1.200,00
```

Interfaces comuns da Java API

- Permitem que você use suas próprias classes dentro das estruturas fornecidas pelo java.
  - Comparable
    - permite que objetos que a impelementam sejam comparados entre si
  - Serializable
    - utilizada para identificar classes cujos objetos podem ser gravados (serializados) ou lidos

Interfaces comuns da Java API

- Runnable
  - Implementada por qualquer classe da qual objetos devem ser capazes de executar em paralelo
- SwingConstants
  - Contem um conjunto de constantes utilizadas em programação de interfaces graficas com usuários (GUIs) para posicionar elementos na tela

Exercícios